

```
!pip install transformers[torch]
!pip install datasets
!pip install transformers
!pip install sentence_transformers
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from transformers[torch]) (2.31.0)
Requirement already satisfied: tokenizers<0.19,>=0.14 in /usr/local/lib/python3.10/dist-packages (from transformers[torch])
Requirement already satisfied: safetensors>=0.3.1 in /usr/local/lib/python3.10/dist-packages (from transformers[torch]) (0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.10/dist-packages (from transformers[torch]) (4.66.1)
Requirement already satisfied: torch!=1.12.0,>=1.10 in /usr/local/lib/python3.10/dist-packages (from transformers[torch])
Collecting accelerate>=0.20.3 (from transformers[torch])
  Downloading accelerate-0.25.0-py3-none-any.whl (265 kB)
    265.7/265.7 kB 2.7 MB/s eta 0:00:00
Requirement already satisfied: psutil in /usr/local/lib/python3.10/dist-packages (from accelerate>=0.20.3->transformers[torch])
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub)
Requirement already satisfied: sympy in /usr/local/lib/python3.10/dist-packages (from torch!=1.12.0,>=1.10->transformers[torch])
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torch!=1.12.0,>=1.10->transformers[torch])
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.10/dist-packages (from torch!=1.12.0,>=1.10->transformers[torch])
Requirement already satisfied: triton==2.1.0 in /usr/local/lib/python3.10/dist-packages (from torch!=1.12.0,>=1.10->transformers[torch])
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->transformers[torch])
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->transformers[torch])
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->transformers[torch])
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->transformers[torch])
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from Jinja2->torch!=1.12.0,>=1.10->transformers[torch])
Requirement already satisfied: mpmath>=0.19 in /usr/local/lib/python3.10/dist-packages (from sympy->torch!=1.12.0,>=1.10->transformers[torch])
Installing collected packages: accelerate
Successfully installed accelerate-0.25.0
Collecting datasets
  Downloading datasets-2.16.1-py3-none-any.whl (507 kB)
    507.1/507.1 kB 4.0 MB/s eta 0:00:00
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from datasets) (3.13.1)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from datasets) (1.23.5)
Requirement already satisfied: pyarrow>=8.0.0 in /usr/local/lib/python3.10/dist-packages (from datasets) (10.0.1)
Collecting pyarrow-hotfix (from datasets)
  Downloading pyarrow_hotfix-0.6-py3-none-any.whl (7.9 kB)
Collecting dill<0.3.8,>=0.3.0 (from datasets)
  Downloading dill-0.3.7-py3-none-any.whl (115 kB)
    115.3/115.3 kB 16.2 MB/s eta 0:00:00
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from datasets) (1.5.3)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.10/dist-packages (from datasets) (2.31.0)
Requirement already satisfied: tqdm>=4.62.1 in /usr/local/lib/python3.10/dist-packages (from datasets) (4.66.1)
Requirement already satisfied: xxhash in /usr/local/lib/python3.10/dist-packages (from datasets) (3.4.1)
Collecting multiprocess (from datasets)
  Downloading multiprocess-0.70.15-py310-none-any.whl (134 kB)
    134.8/134.8 kB 18.1 MB/s eta 0:00:00
Requirement already satisfied: fsspec[http]<=2023.10.0,>=2023.1.0 in /usr/local/lib/python3.10/dist-packages (from dataset)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.10/dist-packages (from datasets) (3.9.1)
Requirement already satisfied: huggingface-hub>=0.19.4 in /usr/local/lib/python3.10/dist-packages (from datasets) (0.19.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from datasets) (23.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from datasets) (6.0.1)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (23.1.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (6.0.2)
Requirement already satisfied: yarl<2.0,>=1.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.9.4)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.4.0)
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.3.1)
Requirement already satisfied: async-timeout<5.0,>=4.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (4.0.3)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests>=2.19.0->datasets)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests>=2.19.0->datasets)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests>=2.19.0->datasets)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests>=2.19.0->datasets)
Requirement already satisfied: python-dateutil>=2.8.1 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2.8.2)
```

Load data from Excel file

```
import pandas as pd
from datasets import Dataset
```

```
from google.colab import drive
```

```
# Mount Google Drive
drive.mount('/content/drive')
```

```
hc = pd.read_csv('/content/drive/MyDrive/hcV3-stories.csv', sep=',', encoding = "ISO-8859-1")
```

```
Mounted at /content/drive
```

```
hc.head()
```

	AssignmentId	WorkTimeInSeconds	WorkerId	annotatorAge	annotatorGender	annotatorRace	dist
0	32RIADZISTQWI5XIVG5BN0VMYFRS4U	1641	E9TY34YY	25.0	man	white	
1	3018Q3ZVOJCZJFDMPSFXATCQ4DARA2	1245	237K2NI1	25.0	woman	white	
2	3IRIK4HM3B6UQBC0HI8Q5TBJZLEC61	1159	FK5QTANB	35.0	woman	black	
3	3018Q3ZVOJCZJFDMPSFXATCQG04RAI	500	UYOSBBRS	30.0	woman	white	
4	3MTMREQS4W44RBU8OMP3XSK8NMJAWZ	1074	34BFLNJV	25.0	man	white	

5 rows × 23 columns

Here, we change values of memType to numeric and change the names of story and memType to sent and labels.

```
mem = hc[hc['memType']!='retold']
mem = mem.dropna(subset=['story', 'memType'])
mem = mem[['story', 'memType']]
mem['memType'][mem['memType']=='recalled']= 0
mem['memType'][mem['memType']=='imagined']= 1
mem = mem.rename(columns={'story': 'sent', 'memType': 'labels'})
mem['type']='B'
```

mem.head()

		sent	labels	type
0	Concerts are my most favorite thing, and my bo...		1	B
1	The day started perfectly, with a great drive ...		0	B
2	It seems just like yesterday but today makes f...		1	B
3	Five months ago, my niece and nephew were born...		0	B
4	About a month ago I went to burning man. I was...		1	B

✧ Tokenizer and Model

```
class CustomDataset(Dataset):
    def init(self, data, labels, transform=None):
        self.data = data
        self.labels = labels
        self.transform = transform

    def len(self):
        return len(self.data)

    def getitem(self, idx):
```

```

sample = {'data': self.data[idx], 'labels': self.labels[idx]}

if self.transform:
    sample = self.transform(sample)

return sample

```

✓ Preprocessing

Here we also drop the rows with very short stories (stories with less than 60 words).

```

from torch.utils.data import Dataset

# Print columns before cleaning
print("Columns before cleaning:", mem.columns)

# Clean column names
mem.columns = mem.columns.str.strip().str.normalize('NFKD').str.encode('ascii', errors='ignore').str.decode('utf-8')

print("Columns after cleaning:", mem.columns)

# Check the DataFrame structure
print("DataFrame structure:")
print(mem.head())

# Drop rows where any of the specified columns have NaN values
mem = mem.dropna()
# see if there is any NA value existing
mem.isna().sum()
mem.shape[0]

mem['word_count'] = mem['sent'].apply(lambda x: len(str(x).split()))
mem = mem[mem['word_count'] >= 60]

# Drop the 'word_count' column as it was used for filtering
mem = mem.drop(columns=['word_count'])

# Print the resulting DataFrame
print("DataFrame after filtering:")
print(mem.head())
mem.shape[0]

```

Columns before cleaning: Index(['sent', 'labels', 'type'], dtype='object')

Columns after cleaning: Index(['sent', 'labels', 'type'], dtype='object')

DataFrame structure:

	sent	labels	type
0	Concerts are my most favorite thing, and my bo...	1	B
1	The day started perfectly, with a great drive ...	0	B
2	It seems just like yesterday but today makes f...	1	B
3	Five months ago, my niece and nephew were born...	0	B
4	About a month ago I went to burning man. I was...	1	B

DataFrame after filtering:

	sent	labels	type
0	Concerts are my most favorite thing, and my bo...	1	B
1	The day started perfectly, with a great drive ...	0	B
2	It seems just like yesterday but today makes f...	1	B
3	Five months ago, my niece and nephew were born...	0	B
4	About a month ago I went to burning man. I was...	1	B

5450

```

def create_dataset(Scenario={'train': [], 'test': [], 'scenario': 1}, mem=None, seed=42, cv=5):
    train = {f'split_{i + 1}': [] for i in range(cv)}
    test = {f'split_{i + 1}': [] for i in range(cv)}

    np.random.seed(seed)

    num_row = (mem.index.shape[0] - (mem.shape[0] % 10))
    me_arr = np.random.choice(mem.index.shape[0], int(num_row), replace=False)
    me_arr_r = me_arr.reshape((cv, int(num_row / cv)))

    cv_ind = 0
    for k in train.keys():
        if 'hyppocampus' in Scenario['train']:
            ind = me_arr_r[cv_ind, :]
            antind = [i for i in me_arr if i not in ind]
            mem_train = mem[mem.index.isin(antind)]
            mem_test = mem[mem.index.isin(ind)]

            train[k].append(mem_train)

```

```

        test[k].append(mem_test)

    train[k] = pd.concat(train[k])
    test[k] = pd.concat(test[k])

    cv_ind += 1

# return should be outside the loop
return train, test

```

```

# Funzione di preprocessing per il tuo dataset
def preprocess_function(examples):
    return tokenizer(examples["sent"], truncation=True, padding=True)

```

Experiment Scenario 1

model

Here we used GPT2ForSequenceClassification to do classification with GPT2.

```

from transformers import GPT2Tokenizer, GPT2ForSequenceClassification, TrainingArguments, Trainer
from datasets import Dataset
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
from torch.utils.data import DataLoader
import numpy as np
import pandas as pd
import torch
from tqdm.auto import tqdm

# Configurazione iniziale
seed = 42
np.random.seed(seed)
torch.manual_seed(seed)
numcv = 5
num_epochs = 3
model_size = 'gpt2'
Scenario = {'train': ['hyppocorpus'], 'test': ['hyppocorpus'], 'scenario': 1}

# Caricamento e preparazione dei dati
train, test = create_dataset(Scenario, mem, seed, numcv)

# Inizializzazione del tokenizer e del modello
tokenizer = GPT2Tokenizer.from_pretrained(model_size)
tokenizer.pad_token = tokenizer.eos_token # Imposta il token di padding
model = GPT2ForSequenceClassification.from_pretrained(model_size, num_labels=2)
model.config.pad_token_id = tokenizer.pad_token_id # Aggiorna il modello con il token di padding

results = {}

for sp in tqdm(train.keys()):
    if not isinstance(train[sp], pd.DataFrame):
        print(f"Error: train[{sp}] is not a DataFrame. It's a {type(train[sp])}.")
        continue
    # Preparazione del dataset
    data_train = Dataset.from_pandas(train[sp]).map(preprocess_function, batched=True)
    data_train.set_format(type='torch', columns=['input_ids', 'attention_mask', 'labels'])

    data_test = Dataset.from_pandas(test[sp]).map(preprocess_function, batched=True)
    data_test.set_format(type='torch', columns=['input_ids', 'attention_mask', 'labels'])

    # Configurazione degli argomenti di training
    training_args = TrainingArguments(
        output_dir=f"{model_size}_scenario_{Scenario['scenario']}",
        evaluation_strategy='epoch',
        learning_rate=5e-5,
        per_device_train_batch_size=2,
        weight_decay=0.01,
        save_total_limit=3,
        num_train_epochs=num_epochs,
        fp16=False,
        push_to_hub=False,
    )

    # Inizializzazione del Trainer

```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=data_train,
    eval_dataset=data_test,
    tokenizer=tokenizer
)

# Training del modello
trainer.train()

# Funzione per calcolare le previsioni
def compute_predictions(model, data_loader):
    model.eval()
    predictions = []
    actuals = []

    with torch.no_grad():
        for batch in data_loader:
            # Ora 'batch' è un dizionario
            input_ids = batch['input_ids'].to(model.device)
            attention_mask = batch['attention_mask'].to(model.device)
            labels = batch['labels'].to(model.device)

            outputs = model(input_ids, attention_mask=attention_mask)
            logits = outputs.logits
            preds = torch.argmax(logits, dim=-1)

            predictions.extend(preds.cpu().numpy())
            actuals.extend(labels.cpu().numpy())

    return predictions, actuals

# Preparazione del DataLoader di test
test_loader = DataLoader(data_test, batch_size=2)

# Calcolo delle previsioni
predictions, actual_labels = compute_predictions(model, test_loader)

# Calcolo dell'accuratezza
accuracy = accuracy_score(actual_labels, predictions)
print(f'Accuracy for {sp}: {accuracy}')

# Salvataggio dei risultati
results[sp] = accuracy

# Calcolo dell'accuratezza media
average_accuracy = np.mean(list(results.values()))
print('Average accuracy:', average_accuracy)
```

vocab.json: 100% 1.04M/1.04M [00:00<00:00, 14.2MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 10.4MB/s]
tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 5.80MB/s]
config.json: 100% 665/665 [00:00<00:00, 41.4kB/s]
model.safetensors: 100% 548M/548M [00:03<00:00, 192MB/s]
Some weights of GPT2ForSequenceClassification were not initialized from the model checkpoint at gpt2 and are newly initialized. You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
100% 5/5 [3:08:45<00:00, 2276.35s/it]

Map: 100% 3453/3453 [00:08<00:00, 399.19 examples/s]

Map: 100% 854/854 [00:01<00:00, 488.11 examples/s]

 [5181/5181 37:03, Epoch 3/3]

Epoch	Training Loss	Validation Loss
1	0.867400	0.955267
2	0.959900	1.185419
3	0.652400	1.286596

Accuracy for split_1: 0.7494145199063232

Map: 100% 3446/3446 [00:08<00:00, 436.92 examples/s]

Map: 100% 861/861 [00:01<00:00, 490.07 examples/s]

 [5169/5169 34:41, Epoch 3/3]

Epoch	Training Loss	Validation Loss
1	1.072900	0.884471
2	0.732900	0.818571
3	0.493400	0.937780

Accuracy for split_2: 0.851335656213705

Map: 100% 3445/3445 [00:06<00:00, 502.83 examples/s]

Map: 100% 862/862 [00:02<00:00, 417.64 examples/s]

 [5169/5169 37:20, Epoch 3/3]

Epoch	Training Loss	Validation Loss
1	0.853600	0.498725
2	0.476400	0.596340
3	0.337800	0.538087

Accuracy for split_3: 0.825754060321026

Start coding or [generate](#) with AI.

Map: 100% 858/858 [00:01<00:00, 494.30 examples/s]

 [5175/5175 37:25, Epoch 3/3]

Epoch	Training Loss	Validation Loss
1	0.582100	0.184048
2	0.353900	0.161997
3	0.176200	0.155487

Accuracy for split_4: 0.9766899766899767

Map: 100% 3435/3435 [00:07<00:00, 458.50 examples/s]

Map: 100% 872/872 [00:01<00:00, 494.30 examples/s]

 [3437/5154 24:02 < 12:01, 2.38 it/s, Epoch 2/3]

Epoch	Training Loss	Validation Loss
1	0.414500	0.113312